ACCENTURE 2026–2027 HIRING · TECHNICAL ASSESSMENT · SECTION B

Pseudocode + OOPs Complete Guide

> ≈ 10 of 45 questions · trace code in your head · no pen & paper_

The hardest section of the Technical Assessment, explained from zero: pseudocode syntax, operators and precedence traps, number systems, bitwise tricks, loops, recursion, arrays and strings, the patterns behind common code, and complete OOPs. Includes 59 practice MCQs whose answers come from actually running the code in C and Java.

Operators & traps | Bitwise | Loops | Recursion | Arrays & strings | OOPs

≈ 10	~90s	59	gcc + JDK
Questions in the test	Per question	Practice MCQs	Answers verified by execution

Independent preparation guide · Not affiliated with or endorsed by Accenture

Edition: 2026–2027

INSIDE THIS GUIDE

Contents

00 What's tested and the strategy

P1 The pseudocode language

P2 Operators, precedence and traps

P3 Number systems and two's complement

P4 Bitwise operations in depth

P5 Conditions and switch

P6 Loops and tracing

P7 Functions and recursion

P8 Arrays and strings

P9 "What does this code do?" patterns

01 Classes, objects, constructors

02 Encapsulation and access modifiers

03 Abstraction: abstract classes and interfaces

04 Inheritance and constructor chaining

05 Polymorphism: overloading and overriding

06 static, final, relationships and more

✓ Practice sets and answer keys

Z 7-day plan and cheat sheet

KNOW THE ENEMY

00 What's Tested & the Strategy

About **10 of the 45** technical questions come from here. Most show a **10–12 line program** and ask for its output, and a few ask about OOPs concepts. You get **no pen and paper**, so everything is traced in your head. Candidates call it the trickiest part of the test.

Question type	What they ask	Covered in
Output of code (most)	"What will be printed?", using loops, conditions, recursion and arrays	P5–P9
Bitwise logic	& | ^ ~ << >> on small numbers, often inside loops	P3–P4
Operator traps	Precedence, ++/--, integer division, mod with negatives, short-circuit	P2
Iteration counts	"How many times does the loop run?" or "What's the final value of i?"	P6
OOPs concept / output	The pillars, overloading vs overriding, constructor order, static, access	O1–O6

🧠 Mental trace table

Keep only the variables that change in your head, and update them line by line. Section P6 shows how.

🔍 Spot the pattern

Most snippets are disguised classics: digit sum, reverse, GCD, counting bits. Recognise one and you're done in seconds (P9).

⏱️ 2-minute rule

If a trace passes 2 minutes, eliminate the options you can rule out, pick the best one and move on.

PART 1 · PSEUDOCODE

Reading & Tracing Code

Pseudocode is language-neutral, but it **behaves like C or Java**: integers truncate, characters are numbers, arrays start at 0, and operators follow C precedence. Learn these rules once and every question becomes mechanical.

P1

SYNTAX YOU'LL SEE

The Pseudocode Language

Construct	Example	Meaning / rule
Declaration	`Integer a, b = 5` · `Float f` · `Char c`	Integer holds whole numbers only
Assignment	`Set a = b + 2` or `a = b + 2`	The right side is evaluated first, then stored
Input / output	`Read n` · `Print a, b`	Usually printed with spaces between values
Condition	`if (…) … else if (…) … else … end if`	Only the first true branch runs
Counting loop	`for (i = 1 to 5)`	**Inclusive** on both ends: runs 5 times
C-style loop	`for (i = 0; i < 5; i = i + 1)`	Runs while the condition is true: i = 0…4
while / do-while	`while (c) … end while` · `do … while (c)`	do-while runs **at least once**
Jump	`break` · `continue`	Leave the innermost loop · skip to the next pass
Function	`Function f(Integer n) … return x … End Function`	Parameters are passed **by value** unless stated
Arrays	`Integer a[5]` → a[0]…a[4]	0-indexed. a[5] is out of bounds
Operators	`mod` (or %), `&& || !`, `& | ^ ~ << >>`	As in C/Java

Integer division
`7 / 2 = 3`, `2 / 5 = 0`, `-7 / 2 = -3` (truncates toward zero). If either side is a Float, the result keeps decimals: `7.0 / 2 = 3.5`.

Characters
A Char is a small integer (its ASCII code): `'A' = 65`, `'a' = 97`, `'0' = 48`. `'A' + 1` = 66, which is 'B'. `'a' - 'A'` = 32.

Integer storage
An Integer has limits (32-bit: about ±2.1 billion). Going past the maximum **wraps around** to negative (overflow). Assigning a decimal to an Integer **truncates**: `Integer x = 9.8` gives 9.

P2

WHERE MOST MARKS ARE LOST

Operators, Precedence & Traps

Level	Operators (highest first)	Associativity	Example
1	`() [] .` and postfix `x++ x--`	left → right	—
2	Unary `++x --x + - ! ~`, (type) casts	right → left	`-~5` = 6
3	`* / %`	left → right	`8 / 2 * 4` = 16
4	`+ -`	left → right	`10 - 4 - 3` = 3
5	`<< >>`	left → right	`1 << 2 + 1` = 8
6	`< <= > >=`	left → right	—
7	`== !=`	left → right	`3 < 2 == 0` → (3 < 2) == 0 = 1
8	`&`	left → right	`x & 1 == 0` → x & (1 == 0)
9	`^`	left → right	—
10	`|`	left → right	`5 | 2 & 3` = 5 | 2 = 7
11	`&&`	left → right	—
12	`||`	left → right	`a || b && c` → a || (b && c)
13	`?:`	right → left	—
14	`= += -= *= /= %= &= |= ^= <<= >>=`	right → left	`a = b = 5`

Memory trick: *Unary → Arithmetic (* / % then + −) → Shift → Relational → Equality → Bitwise (& ^ |) → Logical (&& ||) → Ternary → Assignment.*

++ and -- (pre vs post)
`b = a++` → b gets the **old** value, then a increases.
`b = ++a` → a increases **first**, then b gets the new value.
With a = 5: `b = a++` gives b = 5, a = 6. `c = ++a` gives c = 7, a = 7.
Avoid reasoning about `a++ + ++a` in one statement. In C that's undefined behaviour, and well-formed exam questions avoid it.

Division & mod with negatives (C/Java)
Division truncates toward 0: `-7 / 2 = -3`, `7 / -2 = -3`.
Mod takes the sign of the **dividend** (the left side): `-7 % 2 = -1`, `7 % -2 = 1`.
Check: `(a / b) * b + a % b == a`.
Excel's MOD(−7, 3) = 2 and Python's −7 % 3 = 2 follow the divisor's sign instead. Pseudocode follows C/Java.

Short-circuit evaluation
In `A && B`, if A is false, **B is never evaluated**.
In `A || B`, if A is true, B is never evaluated.
This matters when B contains `++j` or a function call: those side effects might never happen.

Truth values
In C-style pseudocode, any **non-zero** value is true and 0 is false. Comparisons give 1 or 0:
`(5 > 3) + (2 > 9)` = 1 + 0 = **1**. `!5` = 0, `!0` = 1.

Top 6 operator traps: ① `x << 1 + 2` means x << 3 · ② `x & 1 == 0` means x & 0 · ③ `=` inside if assigns (`if (a = 0)` is always false) · ④ integer `/` drops decimals before you multiply: `(1 / 2) * 10` = 0 · ⑤ the sign of a negative mod · ⑥ skipped side effects in && and ||.

P3

BINARY THINKING

Number Systems & Two's Complement

Decimal → binary (place-value method)

n	64	32	16	8	4	2	1
13	0	0	0	1	1	0	1
21	0	0	1	0	1	0	1
58	0	1	1	1	0	1	0
100	1	1	0	0	1	0	0

Subtract the largest power of 2 that fits, mark a 1, and repeat. Or divide by 2 repeatedly and read the remainders from bottom to top.

Powers of 2 to memorise

$2^0 = 1$	$2^1 = 2$	$2^2 = 4$	$2^3 = 8$
$2^4 = 16$	$2^5 = 32$	$2^6 = 64$	$2^7 = 128$
$2^8 = 256$	$2^9 = 512$	$2^{10} = 1024$	$2^{16} = 65536$

Octal groups bits in 3s and **hex** groups bits in 4s: 58 = 111 010 = **72_8** = 0011 1010 = **$3A_{16}$**. In code, `0x` marks hex and a leading `0` marks octal in C (`010` = 8!).

Negative numbers: two's complement
To get −x: **invert all the bits of x, then add 1**. In 8 bits, 5 = 00000101 → inverted 11111010 → +1 = **11111011** = −5. The leftmost bit is the sign (1 = negative). An 8-bit signed range is −128 to +127.
That's why `~x = -(x + 1)` : inverting the bits is "−x − 1". So `~5 = -6`, `~0 = -1` and `~(-3) = 2`.

P4

THE #1 TOPIC

Bitwise Operations in Depth

Op	Name	Rule (bit by bit)	12 op 10 → (1100, 1010)
`&`	AND	1 only if **both** are 1	1000 = **8**
`|`	OR	1 if **either** is 1	1110 = **14**
`^`	XOR	1 if the bits are **different**	0110 = **6**
`~`	NOT	Flip every bit (= −x − 1)	~12 = **−13**
`<< n`	Left shift	Moves bits left and fills with 0 (= ×2^n)	12 << 2 = **48**
`>> n`	Right shift	Moves bits right (= ÷2^n, rounding down for positives)	12 >> 2 = **3**

Trick	Code	Example
Is x odd?	`x & 1` (1 = odd)	7 & 1 = 1 · 10 & 1 = 0

Trick	Code	Example
Is bit k set?	`(x >> k) & 1`	(13 >> 2) & 1 = 1
Set / clear / toggle bit k	`x | (1<<k)` · `x & ~(1<<k)` · `x ^ (1<<k)`	8 | 2 = 10 · 15 & ~4 = 11 · 5 ^ 1 = 4
Clear the lowest 1-bit	`x & (x − 1)`	12 & 11 = 8
Keep only the lowest 1-bit	`x & (−x)`	12 & −12 = 4
Power of 2? (x > 0)	`(x & (x − 1)) == 0`	16 → yes · 12 → no
Count the 1-bits	loop `x = x & (x − 1)` and count	58 → 4
Swap without a temp variable	`a^=b; b^=a; a^=b`	Works because x ^ x = 0 and x ^ 0 = x
Same or opposite signs?	`(a ^ b) < 0` → opposite	5 ^ −3 < 0 → true

Identities worth knowing: `a + b = (a ^ b) + 2·(a & b)` · `a | b = (a ^ b) + (a & b)` · `a ^ a = 0` · `a ^ 0 = a` · `~~a = a` · XOR is associative and commutative, so the order doesn't matter.

P5

BRANCHING

Conditions & switch

```
Integer a = 5, b = 10
if (a > 3)
    if (b < 5) Print "X"
else Print "Y"   // pairs with the INNER if → prints Y
```

Dangling else

An `else` always belongs to the **nearest** unmatched `if`, whatever the indentation. Indentation is just decoration.

```
switch (k)            // k = 2
  case 1: s = s + 1
  case 2: s = s + 2    // enters here
  case 3: s = s + 3    // falls through!
          break
  default: s = 0
```

switch fall-through

Execution starts at the matching case and **keeps going** through the cases below until it reaches a `break`. With k = 2 and s = 0 at the start, s = 2 + 3 = **5**. `default` runs only when nothing matches, or when it's reached by falling through.

Condition traps: `if (x = 5)` assigns 5, which is non-zero, so it's always true · `if (a > b > c)` compares `(a > b)`, which is 0 or 1, against c · a semicolon right after `if (…);` makes an empty statement, so the next line always runs · in an **else-if chain**, only the first true branch runs.

P6

REPETITION

Loops & Tracing

The mental trace table: the core technique

1. List only the variables that **change** inside the loop.
2. Write the values **before** the loop (pass 0).
3. For each pass: check the condition → run the body line by line → update the counter.
4. After 2–3 passes, **look for a pattern** (doubling, a running sum, alternation) and jump to the end.

```
Integer i = 1, s = 0
while (i <= 20)
    s = s + i
    i = i * 3
end while
Print s, i
```

Pass	check i ≤ 20	s	i
0	–	0	1
1	1 ✓	1	3
2	3 ✓	4	9
3	9 ✓	13	27
4	27 ✗ stop	Output: **13 27**	

Loop shape	Number of iterations
`for (i = a to b)`	b – a + 1 (0 if b < a)
`for (i = 0; i < n; i++)`	n · afterwards i = n
`for (i = 0; i < n; i += k)`	⌈n / k⌉
`for (i = 1; i < n; i *= 2)`	$\lceil \log_2 n \rceil$ (i = 1, 2, 4…)
`for i = 1 to n` containing `for j = 1 to n`	n^2
`for i = 1 to n` containing `for j = 1 to i` (or `j = i to n`)	n(n + 1)/2
`for i = 1 to n` containing `for j = i+1 to n`	n(n − 1)/2 (pairs)
Three nested loops `1..n, 1..i, 1..j`	n(n + 1)(n + 2)/6

Loop traps: off-by-one (`<` vs `<=`) · the value of the counter **after** the loop (one past the last value) · `break` leaves only the **innermost** loop · `continue` in a for-loop still runs the update step · changing the loop variable inside the body · a **semicolon after** `for(…);` gives an empty body that runs once in total.

P7

FUNCTIONS CALLING THEMSELVES

Functions & Recursion

Every recursive function needs (1) a **base case** that stops it and (2) a **recursive case** that moves toward the base case. Each call gets its **own copy** of its local variables, stored on the call stack. Without a base case you get infinite recursion and a **stack overflow**.

```
Function f(Integer n)
    if (n == 0) return 0
    return n mod 10 + f(n / 10)
End Function
Print f(472)
```

Going down (calls)	Coming back (returns)
f(472) = 2 + f(47)	2 + 11 = **13**
f(47) = 7 + f(4)	7 + 4 = 11
f(4) = 4 + f(0)	4 + 0 = 4
f(0) → base case	0

Head vs tail printing
`Print n; f(n−1)` → prints on the way **down**: 3 2 1
`f(n−1); Print n` → prints on the way **up**: 1 2 3
With both: 3 2 1 1 2 3.

Branching recursion (two calls)
Draw a **recursion tree**. The number of calls in naive Fibonacci is C(n) = C(n−1) + C(n−2) + 1: C(5) = 15 and C(10) = 177. The values repeat, which is why memoisation helps.

Recognise this recursion...	...it computes
`n * f(n−1)`, f(0) = 1	n! (factorial): f(5) = 120
`f(n−1) + f(n−2)`	Fibonacci: 0 1 1 2 3 5 8 13...
`n % 10 + f(n / 10)`	Sum of digits
`1 + f(n / 10)`	Number of digits
`g(b, a % b)`, g(a, 0) = a	GCD (Euclid). LCM = a·b / GCD
`a * p(a, b−1)`, or squaring	Power a^b
`a + g(a, b−1)`	Multiplication a × b
`n % 2 + f(n / 2)`	Number of 1-bits in n
`f(n / 2)` then `Print n % 2`	Prints n in binary
`rev(n/10, r*10 + n%10)`	Reverses the digits

Call by value vs reference
By value: the function gets a copy, so changes don't affect the caller (a swap(a, b) by value does nothing). **By reference / pointer**: changes affect the caller. In Java, primitives are copied, and for objects **the reference is copied**: the function can change the object's fields, but can't make the caller's variable point somewhere else.

Static / global variables in recursion are **shared** by all calls, unlike local variables. If a function does `count = count + 1` on a global, count ends up as the **total number of calls**.

P8

COLLECTIONS

Arrays & Strings

Arrays
Fixed size, contiguous memory, 0-indexed. `a[i]` is O(1). The last index is **n − 1**. **2D arrays** are stored row by row (row-major): m[i][j] is row i, column j. The main diagonal is m[i][i] and the anti-diagonal is m[i][n−1−i].

Strings
A sequence of characters, with s[0] first. In C a string ends with `'\0'`, so `char s[6]` holds 5 letters. In Java, Strings are **immutable**, so methods return new strings.

Pattern	Code idea	Effect
Running max / min	`m = a[0]; if (a[i] > m) m = a[i]`	Largest element
Reverse in place	`swap(a[i], a[n−1−i])` for i < n/2	Reversed array
Prefix sum	`a[i] = a[i] + a[i−1]`	Running totals
Shift / rotate left	`t = a[0]; a[i] = a[i+1]; a[n−1] = t`	Rotate by 1
Count matches	`if (a[i] == x) c++`	Frequency
One bubble-sort pass	`if (a[i] > a[i+1]) swap`	The largest value moves to the end
Palindrome check	compare `s[i]` with `s[n−1−i]`	Reads the same backwards
Case conversion	`c − 32` (lower → upper), `c + 32`	'a' (97) → 'A' (65)
Digit character → value	`c − '0'`	'7' → 7
Vowel / consonant count	check the character against AEIOU	Counts

ASCII anchors
'0'–'9' = 48–57 · 'A'–'Z' = 65–90 · 'a'–'z' = 97–122 · space = 32

Array traps
Index n is out of bounds · loops from 1 skip a[0] · modifying the array while reading it changes later values (prefix sums!)

String traps
Length vs last index (n − 1) · case sensitivity ('A' ≠ 'a') · in Java, `==` compares references and `equals()` compares content

P9

RECOGNISE, DON'T TRACE

"What Does This Code Do?" Patterns

If the loop does this...	...the program computes
`s += n % 10; n /= 10`	Sum of digits
`r = r*10 + n % 10; n /= 10`	Reverse of the number (compare with the original → palindrome check)
`s += (n%10)³ …; compare s with n`	Armstrong number check (153 = $1^3 + 5^3 + 3^3$)
`for d = 2 while d*d ≤ n: if n % d == 0 → not prime`	Prime test
`sum of d where n % d == 0, d < n`, compared with n	Perfect number (6, 28)
`while b ≠ 0: t = b; b = a % b; a = t`	GCD
`c += n & 1; n >>= 1` or `n &= n − 1`	Number of 1-bits
`r += (n % 2) * p; p *= 10; n /= 2`	Binary digits as a decimal-looking number
`f = f * i` for i = 1..n	Factorial
`t = a + b; a = b; b = t`	Fibonacci
`r ^= i` over a range	XOR 1..n follows the cycle n, 1, n+1, 0 by n mod 4

PART 2 · OBJECT-ORIENTED PROGRAMMING

OOPs, Complete

OOPs questions test definitions (the four pillars), rules (what can be overloaded, overridden or inherited) and short Java-like outputs (constructor order, which method runs). Every rule below is shown in a small example.

Class & object | Encapsulation | Abstraction | Inheritance | Polymorphism | static · final

01 THE BASICS

Classes, Objects & Constructors

```
class Student {
    String name;          // attribute
(state)
    int marks;
    Student(String n, int m) {   //
constructor
        this.name = n; this.marks = m;
    }
    boolean passed() { return marks >= 40; }
// method
}
Student s = new Student("Ravi", 72);  //
object
```

Class vs Object

A **class** is a blueprint that defines state (fields) and behaviour (methods). An **object** is an instance created with `new`, which allocates memory on the **heap**. Many objects can come from one class.

Constructor

Same name as the class, **no return type** (not even void), runs automatically when `new` is called. If you write none, Java supplies a **default no-argument constructor**, but only if you write no constructors at all.

Constructor type	Meaning
Default (no-argument)	No parameters. Fields get defaults: 0, false, null
Parameterised	Takes values to initialise the object
Copy constructor	Builds a new object from an existing one (explicit in C++, written by hand in Java)
Overloaded constructors	Several constructors with different parameter lists. `this(…)` calls another one
Destructor	C++ only (`~ClassName()`). Java has **garbage collection** instead

`this` refers to the **current object**. Use it when a parameter has the same name as a field (`this.name = name`), or call `this(…)` from a constructor (it must be the first statement). Without `this`, a parameter **shadows** the field.

02

PILLAR 1

Encapsulation & Access Modifiers

```
class Account {
    private double balance;      // hidden
    public double getBalance() { return
balance; }
    public void deposit(double amt) {
        if (amt > 0) balance += amt;   //
validation
    }
}
```

Encapsulation = bundling data with the methods that operate on it, and **hiding** the data (**data hiding**). Others interact only through public methods, which can validate input. Benefits: control, security, and the freedom to change the internals later.

Modifier (Java)	Same class	Same package	Subclass (other package)	Everywhere
`private`	✓	×	×	×
default (no keyword)	✓	✓	×	×
`protected`	✓	✓	✓	×
`public`	✓	✓	✓	✓

03

PILLAR 2

Abstraction: Abstract Classes & Interfaces

Abstraction means exposing *what* an object does and hiding *how*. You call `car.start()` without knowing about the engine's internals.

```
abstract class Shape {
    abstract double area();        // no body
    void describe() {              // concrete
method
        System.out.print("Area = " + area());
    }
}
class Circle extends Shape {
    double r; Circle(double r){ this.r = r; }
    double area() { return 3.14 * r * r; }
}
```

```
interface Payable {
    int LIMIT = 1000;      // public static
final
    void pay(int amt);     // public abstract
    default void info() {  // Java 8+
        System.out.print("Payable");
    }
}
class Card implements Payable {
    public void pay(int amt) { … }
}
```

	Abstract class	Interface
Keyword	`abstract class` / `extends`	`interface` / `implements`
Methods	Abstract **and** concrete	Abstract, plus default and static (Java 8+) and private (Java 9+)
Fields	Any (instance state allowed)	Only `public static final` constants
Constructor	**Yes** (called via super)	No
Inheritance	A class can extend **one**	A class can implement **many**

	Abstract class	Interface
Can be instantiated?	No	No
Use when	Related classes share code and state ("is-a")	Unrelated classes share a capability ("can-do")

Rules: a class with even one abstract method must be declared abstract · a concrete subclass must implement **all** inherited abstract methods, or be abstract itself · interface methods are implicitly `public`, so the implementing method must also be public.

PILLAR 3

04 Inheritance & Constructor Chaining

Type	Shape	In Java?
Single	A → B	✓
Multilevel	A → B → C	✓
Hierarchical	A → B, A → C	✓
Multiple	A, B → C	Only through **interfaces**
Hybrid	A mix of the above	Through interfaces

Why no multiple class inheritance? The diamond problem. If B and C both override A's `m()` and D extends both, which `m()` does D get? Java forbids it for classes. C++ allows it and solves it with virtual inheritance.

```
class A { A() { print("A "); } }
class B extends A { B() { print("B "); } }
class C extends B { C() { print("C"); } }
new C();   // prints: A B C
```

Constructor chaining

The first line of every constructor is an implicit `super()`. Constructors run **from the top of the hierarchy down**: parent before child. Use `super(args)` to call a parameterised parent constructor. It must be the **first statement**. If the parent has no no-argument constructor, the child **must** call `super(args)`.

What is inherited?

Public and protected members (and default ones in the same package). **Not** private members (they exist but can't be accessed), and **not constructors**.

`super`

`super.method()` calls the parent's version. `super.field` accesses a parent field hidden by the child.

Initialisation order

Static blocks (once, at class load) → instance initialiser blocks → constructor body. Parent before child at each stage.

PILLAR 4

05 Polymorphism: Overloading & Overriding

	Overloading (compile-time / static)	Overriding (run-time / dynamic)
Where	Same class (or inherited)	Parent → child
Method name	Same	Same

	Overloading (compile-time / static)	Overriding (run-time / dynamic)
Parameters	**Must differ** (number, type or order)	**Must be the same**
Return type	Can differ, but a difference in return type **alone isn't enough**	Same, or a subtype (covariant)
Access	Any	Can't be **more restrictive** (public → protected isn't allowed)
Decided by	Compiler, from the argument types	JVM, from the **actual object** type
static / private / final methods	Can be overloaded	Can't be overridden (static ones are *hidden*)

```
class Animal { void sound() { print("..."); }
}
class Dog extends Animal {
    void sound() { print("Woof"); } }
Animal a = new Dog();  // upcasting
(implicit)
a.sound();             // Woof → dynamic
dispatch
```

The golden rule for output questions
Reference type decides what you're *allowed* to call, as well as fields and static methods.
Object type decides which **overridden instance method** actually runs.
So `a.x` uses Animal's field, but `a.sound()` uses Dog's method.

Overload resolution order: exact match → widening of primitives (int → long → float → double) → autoboxing (int → Integer) → varargs. So `f(5.0f)` with only `f(int)`, `f(long)` and `f(double)` available calls **f(double)**.
Downcasting `(Dog) a` must be explicit, and throws `ClassCastException` if the object isn't really a Dog.

KEYWORDS & RELATIONSHIPS

06 static, final, Relationships & More

`static` belongs to the **class**, not to objects. One copy is shared by all objects (counters, constants). Static methods can't use `this` or call instance members directly. Call them as `ClassName.method()`.

`final`: a final variable can't be reassigned · a final method can't be overridden · a final class can't be extended (for example, `String`).

Relationship	Meaning	Example
Inheritance (is-a)	A child is a specialised parent	Dog is an Animal
Association (uses-a)	Objects know or use each other, with independent lifetimes	Teacher ↔ Student
Aggregation (weak has-a)	A whole contains parts that **can exist alone**	Team has Players
Composition (strong has-a)	The parts **die with** the whole	House has Rooms · Car has an Engine

Object class

The root of every Java class. It provides `toString()`, `equals()`, `hashCode()` and `getClass()`.

== vs equals()

`==` compares **references** (same object?). `equals()` compares **content**, if the class overrides it, as String does.

Garbage collection

The JVM frees objects that are no longer referenced. You can't force it: `System.gc()` is only a request.

Coupling & cohesion

Good design has **low coupling** (few dependencies between classes) and **high cohesion** (each class has one focused job).

Exceptions (basics)

`try` → `catch` → `finally`. The finally block **always runs** (even after a return in try). Checked exceptions must be handled or declared.

OOP languages

Java, C++, C#, Python. C is **procedural**, not OOP. "Pure" OOP languages (like Smalltalk) treat everything as an object.

TRACE IN YOUR HEAD

Practice Set 1 · Pseudocode Output

33 questions covering every topic in P1–P9. **Each answer was produced by compiling and running an equivalent C program**, so it matches C/Java behaviour exactly. Trace without paper, aiming for under 90 seconds each.

P1 Precedence — What is the output?

```
Integer a = 10, b = 4, c = 3
Print a - b * c / 2 + a mod c
```

A -1
B 4
C 7
D 5

P2 Shift vs + — What is the output?

```
Integer x = 3
Print x << 1 + 2
```

A 9
B 8
C 12
D 24

P3 & vs == — What is the output?

```
Integer x = 6
if (x & 1 == 0) Print "Even"
else Print "Odd"
```

A Error
B Odd
C Even
D EvenOdd

P4 Pre / post increment — What is the output?

```
Integer a = 5, b, c
b = a++
c = ++a
Print a, b, c
```

A 7 5 6
B 7 6 7
C 6 5 6
D 7 5 7

P5 Compound assignment — What is the output?

```
Integer x = 10
x -= 3
x *= 2
x %= 5
Print x
```

A 2
B 0
C 4
D 14

P6 Negative / and mod — What is the output?

```
Integer a = -7, b = 2
Print a / b, a mod b
```

A -4 -1
B -4 1
C -3 -1
D -3 1

P7 Ternary — What is the output?

```
Integer a = 4, b = 9
Integer m = (a > b) ? a : ((b > 8) ? b - 8 : b)
Print m
```

A 1
B 8
C 9
D 4

P8 Short-circuit — What is the output?

```
Integer i = 0, j = 0
if (i > 0 && ++j > 0) i = 5
if (i == 0 || ++j > 0) i = 7
Print i, j
```

A 7 0
B 5 1
C 7 2
D 7 1

P9 AND / OR / XOR — What is the output?

```
Integer a = 21, b = 13
Print a & b, a | b, a ^ b
```

A 4 30 26
B 5 29 24
C 5 29 8
D 13 21 8

P10 NOT and shifts — What is the output?

```
Integer n = 12
Print (~n) + (n >> 2) + (n << 1)
```

A 14
B 15
C -14
D 38

P11 Kernighan's bit count — What is the output?

```
Integer x = 58, c = 0
while (x != 0)
    x = x & (x - 1)
    c = c + 1
end while
Print c
```

A 6
B 4
C 5
D 3

P12 Power-of-two test — What is the output?

```
Integer cnt = 0
for (i = 1 to 20)
    if ((i & (i - 1)) == 0) cnt = cnt + 1
end for
Print cnt
```

A 5
B 4
C 20
D 10

P13 XOR of a range — What is the output?

```
Integer r = 0
for (i = 1 to 6)
    r = r ^ i
end for
Print r
```

A 21
B 6
C 0
D 7

P14 Decimal → binary digits — What is the output?

```
Integer n = 25, r = 0, p = 1
while (n > 0)
    r = r + (n mod 2) * p
    p = p * 10
    n = n / 2
end while
Print r
```

A 11000
B 25
C 11001
D 10011

P15 Switch fall-through — What is the output?

```
Integer k = 2, s = 0
switch (k)
    case 1: s = s + 1
    case 2: s = s + 2
    case 3: s = s + 3
            break
    case 4: s = s + 4
end switch
Print s
```

A 5
B 2
C 6
D 9

P16 Dangling else — What is the output?

```
Integer a = 5, b = 10
if (a > 3)
    if (b < 5) Print "X"
else Print "Y"
```

A Y
B Nothing
C XY
D X

P17 Triple nested loop — What is the output?

```
Integer c = 0
for (i = 1 to 5)
  for (j = 1 to i)
    for (k = 1 to j)
      c = c + 1
Print c
```

A 125
B 35
C 15
D 55

P18 Multiplicative loop — What is the output?

```
Integer c = 0
for (i = 1; i < 100; i = i * 3)
    c = c + 1
Print c
```

A 4
B 5
C 6
D 33

P19 continue — What is the output?

```
Integer s = 0
for (i = 1 to 10)
    if (i mod 3 == 0) continue
    s = s + i
end for
Print s
```

A 18
B 37
C 36
D 55

P20 Fibonacci-style loop — What is the output?

```
Integer a = 1, b = 1, t
for (i = 1 to 6)
    t = a + b
    a = b
    b = t
end for
Print b
```

A 34
B 13
C 8
D 21

P21 do-while — What is the output?

```
Integer n = 0
do
    n = n + 5
while (n < 0)
Print n
```

A 0
B 5
C -5
D Infinite loop

P22 Recursion print order — What is the output?

```
Function f(Integer n)
    if (n == 0) return
    Print n
    f(n - 1)
    Print n
End Function
f(3)
```

A 1 2 3 3 2 1
B 3 2 1
C 3 2 1 0 1 2 3
D 3 2 1 1 2 3

P23 Counting recursive calls — What is the output?

```
Integer calls = 0
Function fib(Integer n)
    calls = calls + 1
    if (n <= 1) return n
    return fib(n-1) + fib(n-2)
End Function
fib(5)
Print calls
```

A 5
B 15
C 9
D 8

P24 Recursion as multiplication — What is the output?

```
Function g(Integer a, Integer b)
    if (b == 0) return 0
    return a + g(a, b - 1)
End Function
Print g(7, 4)
```

A 74
B 11
C 7
D 28

P25 Accumulator recursion — What is the output?

```
Function rev(Integer n, Integer r)
    if (n == 0) return r
    return rev(n / 10, r * 10 + n mod 10)
End Function
Print rev(1203, 0)
```

A 1203
B 30210
C 3021
D 321

P26 Two-branch recursion — What is the output?

```
Function f(Integer n)
    if (n <= 0) return 1
    return f(n - 1) + f(n - 2)
End Function
Print f(4)
```

A 8
B 13
C 5
D 3

P27 Array reversal — What is the output?

```
Integer a[6] = {1, 2, 3, 4, 5, 6}
for (i = 0 to 2)
    swap(a[i], a[5 - i])
end for
Print a[1] + a[4]
```

A 7
B 5
C 9
D 7.0

P28 Prefix sums — What is the output?

```
Integer a[5] = {2, 4, 1, 3, 5}
for (i = 1 to 4)
    a[i] = a[i] + a[i - 1]
end for
Print a[2] + a[4]
```

A 21
B 22
C 15
D 6

P29 Characters are numbers — What is the output?

```
Char c = 'A'
c = c + 5
Print c
```

A A5
B 70
C E
D F

P30 Vowel counting — What is the output?

```
String s = "PSEUDOCODE"
Integer v = 0
for (i = 0 to length(s) - 1)
    if (s[i] is one of A, E, I, O, U) v = v + 1
end for
Print v
```

A 4
B 10
C 5
D 6

P31 Call by value — What is the output?

```
Function change(Integer x)
    x = x * 2
End Function
Integer a = 5
change(a)
Print a
```

A 5
B 0
C Error
D 10

P32 2D array diagonals — What is the output?

```
Integer m[3][3] = {{1,2,3},{4,5,6},{7,8,9}}
Integer s = 0
for (i = 0 to 2)
    s = s + m[i][i] + m[i][2 - i]
end for
Print s
```

A 45
B 30
C 25
D 15

P33 Prime counting — What is the output?

```
Integer cnt = 0
for (n = 2 to 20)
    Integer p = 1
    for (d = 2; d * d <= n; d = d + 1)
        if (n mod d == 0) p = 0
    cnt = cnt + p
end for
Print cnt
```

A 9
B 10
C 7
D 8

Set 1 · Answer key

Q	Ans	Correct	Why
P1	D	`5`	* / % are evaluated first, left to right: b*c = 12, 12/2 = 6, a mod c = 1. Then 10 − 6 + 1 = 5.
P2	D	`24`	+ binds tighter than <<, so this is x << 3 = 3 × 8 = 24, not (x << 1) + 2.
P3	B	`Odd`	== binds tighter than &, so this is x & (1 == 0) = 6 & 0 = 0, which is false, so the else branch prints Odd even though 6 is even. Write (x & 1) == 0.
P4	D	`7 5 7`	b = a++ uses 5, then a becomes 6. c = ++a makes a 7 first, then uses 7.
P5	C	`4`	10 → 7 → 14 → 14 mod 5 = 4.
P6	C	`-3 -1`	In C and Java, integer division truncates toward zero (−3.5 → −3), and the remainder takes the sign of the dividend: −7 = 2 × (−3) − 1.
P7	A	`1`	a > b is false, so the inner ternary runs: b > 8 is true, so b − 8 = 1.
P8	A	`7 0`	In &&, a false left side skips the right side. In ||, a true left side skips the right side. So ++j never runs, and j stays 0.
P9	B	`5 29 24`	10101 & 01101 = 00101 = 5. | = 11101 = 29. ^ = 11000 = 24.
P10	A	`14`	~12 = −13, 12 >> 2 = 3 and 12 << 1 = 24. −13 + 3 + 24 = 14.
P11	B	`4`	x & (x − 1) clears the lowest 1-bit each pass. 58 = 111010 has four 1s.
P12	A	`5`	Powers of two have exactly one 1-bit, so i & (i − 1) = 0. Up to 20 they are 1, 2, 4, 8 and 16.
P13	D	`7`	1^2 = 3, 3^3 = 0, 0^4 = 4, 4^5 = 1, 1^6 = 7. (XOR from 1 to n repeats with a period of 4.)
P14	C	`11001`	It builds the binary digits as a decimal-looking number: 25 = 11001_2.
P15	A	`5`	It jumps to case 2, and with no break it falls into case 3: 2 + 3 = 5. Then break stops it.
P16	A	`Y`	else always pairs with the NEAREST unmatched if, which is the inner one, whatever the indentation. a > 3 is true and b < 5 is false, so Y prints.
P17	B	`35`	For each i, the inner loops run 1 + 2 + ... + i = i(i + 1)/2 times. 1 + 3 + 6 + 10 + 15 = 35.
P18	B	`5`	i takes the values 1, 3, 9, 27, 81 (243 stops the loop), so 5 iterations.
P19	B	`37`	It adds 1 to 10 but skips the multiples of 3: 55 − (3 + 6 + 9) = 37.
P20	D	`21`	b goes 2, 3, 5, 8, 13, 21.
P21	B	`5`	A do-while loop body always runs at least once, and the condition is checked afterwards.
P22	D	`3 2 1 1 2 3`	Prints before the call appear on the way down (3 2 1). Prints after the call appear on the way back up (1 2 3).
P23	B	`15`	The number of calls is C(n) = C(n−1) + C(n−2) + 1, with C(0) = C(1) = 1. C(5) = 15. That's why naive Fibonacci is $O(2^n)$.
P24	D	`28`	It adds a to itself b times: 7 + 7 + 7 + 7 = 28.
P25	C	`3021`	Each call moves the last digit of n onto r, so the digits come out reversed: 3021.
P26	A	`8`	f(−1) = f(0) = 1, f(1) = 2, f(2) = 3, f(3) = 5, f(4) = 8.
P27	A	`7`	The array becomes {6, 5, 4, 3, 2, 1}, so a[1] = 5 and a[4] = 2, giving 7.

Q	Ans	Correct	Why
P28	**B**	`22`	The running totals are 2, 6, 7, 10, 15. a[2] + a[4] = 7 + 15 = 22.
P29	**D**	`F`	'A' is ASCII 65, and 65 + 5 = 70, which is 'F'.
P30	**C**	`5`	E, U, O, O, E gives 5.
P31	**A**	`5`	A copy of a is doubled inside the function. The caller's a is unchanged.
P32	**B**	`30`	Main diagonal 1 + 5 + 9 plus anti-diagonal 3 + 5 + 7 = 30. The centre 5 is counted twice.
P33	**D**	`8`	It counts the primes from 2 to 20: 2, 3, 5, 7, 11, 13, 17, 19.

OOPS

✓2 Practice Set 2 · OOPs Output & Concepts

J-questions are real Java programs, **run with the JDK to produce the answer key**. T-questions test concepts and rules.

J1 Constructor chaining — What is the output? (Java)

```
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { System.out.print("C"); } }
// inside main():
new C();
```

A A C
B A B C
C C B A
D C

J2 Fields vs methods — What is the output? (Java)

```
class P { int x = 10; void show() { System.out.print("P" + x); } }
class Q extends P { int x = 20; void show() { System.out.print("Q" + x); } }
// inside main():
P obj = new Q();
System.out.print(obj.x + " ");
obj.show();
```

A 20 P20
B 10 P10
C 20 Q20
D 10 Q20

J3 Static method hiding — What is the output? (Java)

```
class P { static void hi() { System.out.print("P"); } }
class Q extends P { static void hi() { System.out.print("Q"); } }
// inside main():
P o = new Q();
o.hi();
```

A Q
B P
C PQ
D Compile error

J4 Overload resolution — What is the output? (Java)

```
static void f(int a) { System.out.print("int "); }
static void f(long a) { System.out.print("long "); }
static void f(double a) { System.out.print("double"); }
// inside main():
f(5); f(5L); f(5.0f);
```

A long long double
B int long double
C int int double
D Compile error

J5 super.method() What is the output? (Java)

```
class A { void m() { System.out.print("A"); } }
class B extends A { void m() { super.m(); System.out.print("B"); } }
// inside main():
new B().m();
```

A AB
B B
C A
D BA

J6 static counter What is the output? (Java)

```
class Counter {
    static int count = 0; int id;
    Counter() { count++; id = count; }
}
// inside main():
Counter c1 = new Counter(), c2 = new Counter(), c3 = new Counter();
System.out.print(c1.id + " " + Counter.count);
```

A 3 3
B 1 1
C 1 3
D 0 3

J7 this vs local What is the output? (Java)

```
class Box {
    int w = 5;
    void set(int w)  { w = w * 2; }
    void set2(int w) { this.w = w * 2; }
}
// inside main():
Box b = new Box();
b.set(10);  System.out.print(b.w + " ");
b.set2(10); System.out.print(b.w);
```

A 5 20
B 20 20
C 10 20
D 5 10

J8 Abstract + polymorphism What is the output? (Java)

```
abstract class Shape { abstract int area(); }
class Sq extends Shape { int s; Sq(int s) { this.s = s; } int area() { return s * s; } }
class Rect extends Shape { int l, b; Rect(int l, int b) { this.l = l; this.b = b; } int
area() { return l * b; } }
// inside main():
Shape[] arr = { new Sq(3), new Rect(2, 5), new Sq(2) };
int t = 0;
for (Shape s : arr) t += s.area();
System.out.print(t);
```

A 13
B 19
C 23
D Compile error

J9 super(arg) What is the output? (Java)

```
class A { A(int x) { System.out.print("A" + x + " "); } }
class B extends A { B() { super(5); System.out.print("B"); } }
// inside main():
new B();
```

A A5 B
B B A5
C Compile error
D B

J10 Interface default method What is the output? (Java)

```
interface I { default String hi() { return "I"; } }
class X implements I { }
class Y implements I { public String hi() { return "Y"; } }
// inside main():
System.out.print(new X().hi() + new Y().hi());
```

A II
B IY
C YY
D Compile error

J11 Passing references What is the output? (Java)

```
class N { int v; }
static void inc(N n) { n.v++; }
static void reset(N n) { n = new N(); n.v = 100; }
// inside main():
N n = new N(); n.v = 1;
inc(n); reset(n);
System.out.print(n.v);
```

A 1
B 100
C 2
D 101

J12 == vs equals What is the output? (Java)

```
// inside main():
String a = new String("hi"), b = new String("hi");
System.out.print((a == b) + " " + a.equals(b));
```

A true false
B true true
C false false
D false true

J13 Initialisation order — What is the output? (Java)

```
class T {
    static { System.out.print("S "); }
    { System.out.print("I "); }
    T() { System.out.print("C "); }
}
// inside main():
new T(); new T();
```

A I C I C
B S I C S I C
C S C I C I
D S I C I C

J14 Upcast and downcast — What is the output? (Java)

```
class A { String n() { return "A"; } }
class B extends A { String n() { return "B"; } String only() { return "only"; } }
// inside main():
A a = new B();
System.out.print(a.n() + " " + ((B) a).only());
```

A A only
B B
C Compile error
D B only

T1 Concept — Two methods differ ONLY in return type, e.g. `int f()` and `double f()`. Result?

A Valid overriding
B Compile error
C Valid overloading
D Runtime error

T2 Concept — Can a constructor be overridden?

A No
B Only if it's public
C Yes, always
D Only in abstract classes

T3 Concept — Can an abstract class have a constructor?

A Yes
B No
C Only if it has no abstract methods
D Only a private one

T4 Concept — By default, variables declared in a Java interface are:

A public static final
B protected
C private
D instance variables

T5 Concept — The 'diamond problem' arises with:

A Method overloading
B Multiple inheritance of classes
C Encapsulation
D Single inheritance

T6 Concept — A class declared `final`:

A Cannot have objects
B Cannot be extended
C Must be abstract
D Cannot have methods

T7 Concept **A Car HAS an Engine, and the Engine is destroyed with the Car. This relationship is:**

A Composition
B Association only
C Inheritance
D Aggregation

T8 Concept **Which call must be the FIRST statement in a constructor if you use it?**

A System.out.print
B return
C new
D super(...) or this(...)

T9 Concept **Private methods of a parent class are:**

A Public in the child
B Overridden automatically
C Not inherited, so they can't be overridden
D Static by default

T10 Concept **Wrapping data with methods and restricting direct access to the data is:**

A Abstraction
B Polymorphism
C Inheritance
D Encapsulation

T11 Concept **Showing only essential features and hiding implementation details is:**

A Overloading
B Encapsulation
C Abstraction
D Inheritance

T12 Concept **Which is decided at RUN time?**

A Which overridden method runs
B Field access through a reference
C Static method calls
D Which overloaded method runs

Set 2 · Answer key

Q	Ans	Correct	Why
J1	B	`A B C`	Every constructor first calls its parent's constructor (an implicit super()), so the order runs from the top down: A, then B, then C.
J2	D	`10 Q20`	Fields are NOT polymorphic: obj.x uses the reference type P, giving 10. Methods ARE polymorphic: show() runs Q's version, which sees Q's x = 20.
J3	B	`P`	Static methods are hidden, not overridden. They're chosen by the reference type (P), not by the object.
J4	B	`int long double`	Java picks the most specific match. A float has no exact match here, so it widens to double.
J5	A	`AB`	super.m() runs the parent's version first, then B prints.
J6	C	`1 3`	A static field is shared by all objects (count = 3). id is per object: c1 got 1.
J7	A	`5 20`	In set(), w is the parameter, which shadows the field, so the field stays 5. this.w refers to the field.

Q	Ans	Correct	Why
J8	C	`23`	Each object runs its own area(): 9 + 10 + 4 = 23. An abstract class can't be instantiated, but its references can point to subclass objects.
J9	A	`A5 B`	super(5) must be the first statement, so the parent constructor runs first.
J10	B	`IY`	X inherits the default method. Y overrides it (it must be public).
J11	C	`2`	Java passes a COPY of the reference. inc changes the shared object, giving 2. reset only re-points its local copy, so the caller's object is untouched.
J12	D	`false true`	== compares references, and these are two different objects. equals() compares content.
J13	D	`S I C I C`	The static block runs once, when the class loads. Then, for every object, the instance block runs before the constructor.
J14	D	`B only`	Upcasting is implicit and keeps the overriding behaviour. To call a B-only method, you must explicitly downcast.
T1	B	`Compile error`	Overloading needs different parameter lists. A different return type alone isn't enough.
T2	A	`No`	Constructors aren't inherited, so they can't be overridden. They can be overloaded.
T3	A	`Yes`	Yes. It runs when a subclass object is created, through super().
T4	A	`public static final`	Interface fields are constants.
T5	B	`Multiple inheritance of classes`	If two parents both define a method, which one does the child get? Java avoids this by forbidding multiple class inheritance.
T6	B	`Cannot be extended`	For example, String is final. A final method can't be overridden, and a final variable can't be reassigned.
T7	A	`Composition`	Composition is a strong has-a. In aggregation (weak has-a), the part can exist independently, like a Team and its Players.
T8	D	`super(...) or this(...)`	You can use only one of them, and it must come first.
T9	C	`Not inherited, so they can't be overridden`	A method with the same name in the child is a new, unrelated method.
T10	D	`Encapsulation`	Use private fields with public getters and setters.
T11	C	`Abstraction`	This is done with abstract classes and interfaces.
T12	A	`Which overridden method runs`	Dynamic dispatch. Overloading, field access and static calls are resolved at compile time.

GET EXAM-READY

7-Day Plan & Cheat Sheet

Day	Plan (60–90 minutes)
Day 1	P1–P2. Learn the precedence table by heart, and do 20 ++/--, division and mod drills.
Day 2	P3–P4. Convert 30 numbers to binary, and practise & | ^ ~ << >> until each takes under 10 seconds.
Day 3	P5–P6. Trace 10 loops **without paper** and memorise the iteration-count formulas.
Day 4	P7–P9. Draw recursion stacks for 6 functions and learn the pattern table.
Day 5	Practice Set 1, timed. For every wrong answer, note which trap caught you.
Day 6	O1–O6. Explain each pillar aloud with an example, then do Practice Set 2.
Day 7	Redo the questions you got wrong, then read the cheat sheet below.

Pseudocode in 12 lines

- Precedence: unary → * / % → + − → << >> → relational → == → & → ^ → | → && → || → ?: → =
- `x << 1 + 2` = x << 3 · `x & 1 == 0` = x & 0
- Integer `/` truncates toward 0 · `%` takes the sign of the left operand: −7 % 2 = −1
- `a++` uses the value then adds · `++a` adds then uses
- && / || short-circuit, so side effects may be skipped
- `~x = −x − 1` · `x<<n = x·2ⁿ` · `x>>n = x/2ⁿ` · `x^x = 0`
- `x & (x−1)` clears the lowest bit · `x & 1` tests odd
- else pairs with the nearest if · switch falls through until a break
- for a to b runs b − a + 1 times · i *= 2 runs $\log_2 n$ times · triangle loops run n(n+1)/2
- Recursion: expand down, combine on the way up · printing after the call gives reverse order
- Naive fib calls: C(n) = C(n−1) + C(n−2) + 1 · globals count calls
- Arrays 0-indexed · 'A' = 65, 'a' = 97, '0' = 48 · call by value doesn't change the caller

OOPs in 12 lines

- Encapsulation = hide data · Abstraction = hide implementation
- Inheritance = is-a · Polymorphism = one interface, many forms
- Overloading: same class, different parameters, compile time · return type alone isn't enough
- Overriding: same signature, child class, run time · can't reduce access
- Fields and static methods follow the reference type · instance methods follow the object type
- Constructors: no return type, not inherited, can't be overridden, can be overloaded
- Constructor order: parent → child · `super()` / `this()` must come first
- Initialisation: static block (once) → instance block → constructor
- Abstract class: can have a constructor and state, extend only one · Interface: constants only, implement many
- Java: no multiple class inheritance (the diamond problem), so use interfaces
- private < default < protected < public · final = no change, override or extend
- Composition = part dies with the whole · Aggregation = part survives · == vs equals()

Exam-day mindset

Read the code **once fully** before tracing, and spot the pattern if there is one. Trace with a mental table, and check the traps: precedence, integer division, ++, short-circuit, else pairing, fall-through. Only then look at the options.

Disclaimer: This is an independent preparation guide for informational purposes only, not affiliated with or endorsed by Accenture. The pseudocode follows common C/Java conventions, and the real test's notation may vary slightly. All questions are original. Code answers were generated by running equivalent C (gcc) and Java (OpenJDK 21) programs.